



DESIGN GUIDE

TinySoul 下一步 设计方向指南

基于深度调研的 Agent 架构演进路径分析

TinySoul Personal Agent Project

2026 年 5 月

目录

一、执行摘要

二、TinySoul 当前架构全景评估

2.1 已完成的七层架构

2.2 核心优势分析

2.3 当前缺失与待探索领域

三、深度调研：Agent 领域五大关键趋势

3.1 趋势一：从文本记忆到命题化记忆系统

3.2 趋势二：Sub-Agent 与分层委托架构

3.3 趋势三：从线性推理到图结构规划

3.4 趋势四：MCP 协议与工具生态标准化

3.5 趋势五：元认知与自演进 Agent

四、推荐的设计方向（按优先级排序）

4.1 P0：跨 Query 记忆系统（Memory Layer）

4.2 P0：INTERNAL_CALL 与 Sub-Agent 委托

4.3 P1：Action 技能图谱与结构化编排

4.4 P1：MCP 协议适配层

4.5 P1：元认知反射与经验积累

4.6 P2：行为草图系统（Behavior Drafts）

五、实施路线图

5.1 Phase 1：记忆层 MVP（4-6 周）

5.2 Phase 2：Sub-Agent 框架（4-6 周）

5.3 Phase 3：编排与生态（6-8 周）

六、技术选型建议

七、风险与挑战

八、结论

参考文献

一、执行摘要

TinySoul 是一个经过多轮迭代构建的个人 Agent 系统，目前已具备稳定的 Query-Loop 核心调度、完善的 Action-State 运行时、多模型 LLM 适配层、OS-Style 异常路由、安全的脚本沙箱执行以及并行调度能力。在同类个人 Agent 项目中，TinySoul 的架构设计展现出罕见的工程成熟度——Action 的元数据与执行分离、实例级注册表消除全局状态、JSON 单一真相源、SignalBus 解耦执行与状态突变等设计决策均体现出深思熟虑的架构权衡。

本报告基于对 TinySoul 全部七份设计文档的深度阅读，结合 2024-2025 年 Agent 架构领域的前沿研究（涵盖记忆系统、Sub-Agent 委托、图结构规划、MCP 协议、元认知自演进等方向），提出以下五个推荐的设计方向，按优先级排列：

表 1 推荐设计方向总览

优先级	方向	核心目标	与 TinySoul 现有架构的契合度
P0	跨 Query 记忆系统	让 Agent 具备对话间持久记忆	高：已有 L0-L4 理论设计
P0	Sub-Agent 委托框架	支持分层任务委托与专业化	高：已有 INTERNAL_CALL 占位
P1	Action 技能图谱	从扁平 Action 到结构化编排	高：已有 profile/cluster/contract
P1	MCP 协议适配层	接入标准化工具生态	中：需新增 adapter 层
P1	元认知反射系统	让 Agent 从经验中学习	高：已有 reasoning action
P2	行为草图系统	支持用户自定义行为模式	中：已有 behavior_drafts 占位

这些方向并非独立实施——它们之间存在深层依赖关系：记忆系统为 Sub-Agent 提供上下文连续性，Sub-Agent 为元认知反射提供多样化的经验来源，MCP 协议为 Action 技能图谱提供外部工具扩展，最终共同构成一个可演进、可扩展、有记忆的 Agent 系统。本报告将对每个方向提供详尽的分析、参考架构和实施建议。

二、TinySoul 当前架构全景评估

在提出下一步设计方向之前，有必要先对 TinySoul 当前架构进行系统性评估，明确已奠定的基础和尚待填补的空缺。

2.1 已完成的七层架构

通过对七份设计文档的通读，TinySoul 当前已形成以下七个高度内聚、职责清晰的模块：

表 2 TinySoul 现有模块及其职责

模块	文件	核心职责	设计亮点
核心调度	core_design_query.md	Action-State-QueryLoop 三元组定义	三步骤固定范式，明确的输入输出契约
Action 系统	action.md	Action 的元数据、执行、注册、超时	Handler/Executor 分离，JSON 单一真相源
Context 模块	context.md	运行时数据持有、序列化、状态管理	Protocol 驱动，Facade + Manager 分离
基础设施	infra.md	配置管理、事件日志、沙箱执行	三层优先级覆盖，四级日志 x 七类事件
LLM 适配层	llm.md	多模型池、故障转移、Prompt 组装	装饰器注册，五元素 Prompt 结构
Loop 调度器	loop.md	主循环调度、并行分发、ONGOING 支持	SignalBus 统一串行/并行路径
异常路由	trap.md	异常捕获、信号路由、状态突变	OS-Style IVT，决策与执行分离

此外，Logico.md 文档展现了项目对 Agent 记忆系统的深层理论思考，提出了基于维特根斯坦《逻辑哲学论》的五层记忆架构（L0 原始记录层至 L4 自我更新层）和命题化记忆模型，这为后续记忆系统的实现提供了坚实的理论基础。

2.2 核心优势分析

TinySoul 的架构在以下方面展现出明显优势：

- 清晰的边界与单向依赖。** Action 框架层对插件层"零感知"，Context 模块完全独立于 Loop，PromptBuilder 不直接操作 LLM——这些设计使得各模块可以独立测试和演进。在调研中发现，许多 Agent 框架（尤其是早期的 LangChain）正是因为模块间耦合过紧而导致维护困难。
- 运行时状态的精心设计。** TodoManager 的双 key 体系（semantic_key / display_key）解决了 LLM 对 todo key 的操作歧义问题；ActionRecord 的 consume-on-read 语义明确区分了"历史记录"和"本轮新事件"；MilestoneManager 的只增设计保证了里程碑的单调性。这些细节体现了对 LLM 交互模式的深入理解。
- 异常处理的系统性。** ErrorTrap 将硬中断（异常）和软中断（Signal）统一为 TrapResult，InterruptHandler 是所有状态突变的唯一执行点，双记录机制确保执行失败既进入 loop_error_list 也进入 action_record_list。这种系统性远超大多数同类项目中将异常处理分散在各处的做法。
- 安全与可控的平衡。** TemporaryScriptExecutor 的六层安全策略（AST 黑名单、内置函数黑名单、模块白名单、受限 builtins、受控文件 I/O、超时控制）提供了一个可执行但受控的代码执行环境，而非简单地将 bash action 暴露给 LLM。

2.3 当前缺失与待探索领域

在文档中被明确标记为未来工作（注释掉的占位或标注 future）的包括：

表 3 TinySoul 当前缺失功能清单

占位符/标记	所在文档	潜在含义
<code>bash action</code> （未注册）	core_design_query.md, action.md	命令执行能力保留但未启用
<code><!--- register_new_action --></code>	core_design_query.md	运行时 Action 注册（临时脚本已支持，但持久化注册未实现）
<code><!--- temporally_registered_ac tions --></code>	core_design_query.md	临时 Action 的生命周期管理

二、TinySoul 当前架构全景评估

<code><!-- memory --></code>	core_design_query.md	记忆系统（跨 Query 持久记忆）
<code><!-- subagent --></code>	core_design_query.md	Sub-Agent / 子代理委托
<code>behavior_drafts(future)</code>	core_design_query.md	行为草图 / 用户自定义行为模式
<code>action_form(discrete/continuous/hybrid)</code>	core_design_query.md	Action 的形式分类（连续 vs 离散）
<code>action_uncertainty(deterministic/stochastic)</code>	core_design_query.md	Action 的不确定性建模

这些占位符构成了 TinySoul 下一步演进的核心线索。它们不是孤立的特性需求，而是指向三个深层能力缺口：**持久记忆**（memory）、**分层委托**（subagent）、**行为编排**（behavior_drafts + action_form + action_uncertainty）。

三、深度调研：Agent 领域五大关键趋势

本章节汇总对 2024-2025 年 Agent 架构领域前沿研究的调研发现，为后续设计方向的论证提供外部参照。

3.1 趋势一：从文本记忆到命题化记忆系统

2024-2025 年，Agent 记忆系统的研究呈现出从"文本片段存储"向"结构化命题记忆"的范式转移。这一趋势与 TinySoul 在 Logico.md 中提出的理论方向高度一致。

MemGPT / Letta (Packer et al., 2023; 2024) 引入了 OS 式的分层记忆架构——将 LLM 的上下文窗口视为"主内存" (RAM)，外部存储视为"磁盘"，通过函数调用实现数据的 page-in/page-out。Letta 后续发展的 Context Repos 支持 Git 风格的版本化记忆。这种分层思想与 TinySoul 当前 action_record_list 作为"运行时记忆"、Workspace 作为"文件系统上下文"的分层有一定对应关系，但缺少跨 Query 的持久记忆层。

A-MEM (Xu et al., 2025) 采用 Zettelkasten 方法，将记忆组织为动态自演进的知识网络——新记忆自动建立与历史记忆的关联链接，并触发对已有记忆的更新。其核心是"原子笔记 + 关系链接 + 自动演化"三元组，这与 Logico.md 中 L1-L3 层的设计（命题抽取、逻辑组合、本体图谱）高度吻合。

Mem0 (Chhikara et al., 2025) 提供了面向生产的记忆层，专注于事实提取和向量化检索，通过提取-更新两阶段流水线自动维护一组高层事实的增删改。其设计强调"减少源头的冗余"，与 Logico.md 中"重言式不应进入长期记忆"的信息增益原则一致。

GRASP (Xia et al., 2026) 虽然是一个技能图谱系统，但其核心发现同样适用于记忆领域：**结构化编排比更大的库更关键**。GRASP 将扁平的技能集转换为带有 precondition-effect 边的类型化 DAG，通过 DAG 编译、节点级验证和局部修复实现比 ReAct 提升 19 分的奖励表现，同时减少 41% 的环境步数。这验证了 Logico.md 中"命题逻辑连接词可以转译为系统中的关系类型"的工程可行性。

关键发现

2024-2025 年的记忆系统研究明确验证了 TinySoul 在 Logico.md 中的理论判断的正确性：Agent 记忆不应是文本仓库，而应是由命题、事实、关系、证据、置信度和语境构成的结构化图像。A-MEM 的 Zettelkasten 实现、Mem0 的事实提取机制、GRASP 的类型化 DAG，都为 TinySoul 的五层记忆架构 (L0-L4) 提供了可落地的工程参考。

3.2 趋势二：Sub-Agent 与分层委托架构

多 Agent 架构在 2024-2025 年经历了从"并行讨论"到"分层委托"的范式演进。

OpenAI Agents SDK (2025) 引入了 `handoff()` 原语，将控制权（而不仅是数据）从 triage agent 转移给 specialist agent。其核心模式是：一个 triage agent 负责路由决策，多个 specialist agent 负责专业执行，handoff 支持双向传递。这一模式被称为"handoff 拓扑"，在 CallSphere 的生产环境中将"无法帮助"的投诉率降低了 71%。

CrewAI (2025) 采用角色驱动的方法，结合 Crew 的协作智能和 Flow 的精确控制。其架构模式包括：顺序流 (Sequential)、层级流 (Hierarchical)、协作/辩论流 (Joint/Debate)。层级流模式——Manager 发布命令，Worker 执行汇报——与 TinySoul 中注释掉的 `subagent` 占位高度相关。

OxyGent (2026) 提出了 Oxy 抽象，将 Agent、工具、流程统一为可互换的原子节点，通过权限驱动的动态规划和实时执行可视化实现模块化。其核心洞察是：需要统一 Agent、工具和流程三个层面的抽象，而非将它们视为不同类别。

关键洞察：最成功的 Sub-Agent 模式不是让多个 Agent 平等讨论，而是建立清晰的**委托边界**——父 Agent 负责任务分解和结果整合，子 Agent 负责专注执行。这与 TinySoul 当前 Action-QueryLoop 的层级结构天然契合：Sub-Agent 可以被视为一种特殊的 Action，其执行不是调用函数而是启动一个独立的 QueryLoop。

3.3 趋势三：从线性推理到图结构规划

Agent 规划能力的研究在 2024-2025 年取得了重要进展，核心趋势是从线性的 Chain-of-Thought 向图结构规划演进。

LATS (Language Agent Tree Search, Zhou et al., 2024) 将蒙特卡洛树搜索 (MCTS) 与 LLM 结合，通过 LM 驱动的价值函数和自我反思实现高效探索。在 HumanEval 上达到 92.7% 的 pass@1 准确率 (GPT-4)。其核心贡献是将推理、行动和规划统一在树搜索框架下。

GRASP (Xia et al., 2026) 代表了这一趋势的最新进展。GRASP 发现：**提供更多技能并不会单调提升性能**——聚焦的 2-3 个技能胜过全面的文档，过多的技能反而有害。瓶颈已从"技能可用性"转向"技能编排"。GRASP 通过四阶段流水线解决这一问题：

1. **记忆条件检索**：融合语义技能匹配与情景经验
2. **DAG 编译**：将检索到的技能组织为带有 precondition-effect 边的类型化图
3. **验证执行与局部修复**：拓扑遍历，节点级前置/后置条件检查，五类修复算子

4. 基于置信度的路由：技能可靠性低时回退到反应式控制

GRASP 在四项交互式基准测试和八个 LLM 骨干上始终优于 ReAct、Reflexion、ExpeL，优势随任务复杂度单调增长。

对 TinySoul 的启示

TinySoul 当前的 Action system 已经具备了 precondition-effect 的元数据定义 (action_contract)，以及 profile 中的 action_intention / action_environment_effect 维度。将 Action 从"扁平列表"升级为"可编译的技能图谱"，引入 DAG 编译层，是顺其自然且价值巨大的演进方向。GRASP 证明这种结构化编排可以将 ReAct 的步数减少 41%，这对降低 LLM 调用成本有直接影响。

3.4 趋势四：MCP 协议与工具生态标准化

Model Context Protocol (MCP) 由 Anthropic 于 2024 年末推出，是 Agent 工具生态领域最重要的标准化事件。2025 年，Anthropic 将 MCP 捐赠给 Linux Foundation 下的 Agentic AI Foundation，OpenAI 和 Google DeepMind 均已采纳。

MCP 的核心架构包含三个角色：

- **Host**：用户面向的 AI 应用（如 Cursor、Claude Desktop）
- **Client**：Host 内部的协议组件，与每个 MCP Server 维护一对一连接
- **Server**：暴露工具、资源和 Prompts 的外部服务

MCP 相对于传统 function calling 的关键创新在于：**协议级解耦**（工具实现与使用分离）、**动态发现与 schema 协商**（运行时枚举工具能力）、**双向通信**（支持工具发起的事件通知）、**访问控制与能力协商**作为一等特性。

对 TinySoul 而言，MCP 的重要性在于：当前 TinySoul 的 Action 体系是封闭注册模型——每个 Action 需要编写 Handler 和 Executor 并在启动时注册到 ActionRegistry。引入 MCP 适配层后，TinySoul 可以作为 MCP Host，动态发现和使用外部 MCP Server 提供的工具，极大地扩展 Agent 的能力边界，而无需为每个新工具编写适配代码。

3.5 趋势五：元认知与自演进 Agent

2025 年，Agent 领域的研究开始关注如何让 Agent 不仅执行任务，还能从经验中学习并自我改进。

MetaAgent (Qian et al., 2025) 提出了"元工具学习" (meta tool learning) 范式：Agent 从最小工作流开始，在解决任务的过程中持续进行自我反思和答案验证，将可操作的见解提炼为简洁文本并动态融入未来任务上下文。MetaAgent 构建了两类经验：

- **自我反思经验** (E_{self})：基于推理轨迹的自我审查，识别认知偏见和可泛化的陷阱
- **验证反思经验** (E_t)：当有 ground-truth 时，比较预测与真实答案，提炼成功/失败的可泛化策略

MetaAgent 在 GAIA、WebWalkerQA 和 BrowseCamp 基准上持续优于工作流基线，并与端到端训练的 Agent 表现相当。

关键机制：MetaAgent 的核心是"动态上下文工程"——将累积的经验 E 注入到每个新任务的输入序列 $X_t = \{q, p, E_{t-1}\}$ 中，使 Agent 在不变更模型参数的情况下持续改进。这与 TinySoul 当前已有但功能单一的 `reasoning` action 形成鲜明对比：`reasoning` 只是单步反思，而 MetaAgent 建立了跨任务的反思-积累-复用闭环。

四、推荐的设计方向（按优先级排序）

基于对 TinySoul 当前架构的评估和上述五大趋势的调研分析，本章节提出六个具体的设计方向，按 P0（立即启动）、P1（Phase 2 启动）、P2（长期规划）三级优先级排列。

4.1 P0：跨 Query 记忆系统（Memory Layer） P0

目标：为 TinySoul 引入跨 Query 持久记忆能力，使 Agent 能够在不同对话间保持对用户偏好、项目上下文和历史决策的记忆。

为什么优先：当前 TinySoul 的 State（QueryState）是随 Query Loop 创建和销毁的纯运行时状态——todo_list、milestone_list、action_record_list 在 loop 结束后全部丢失。这导致 Agent 每次对话都是"从零开始"，无法积累对用户和任务的理解。记忆系统是几乎所有高级 Agent 能力（Sub-Agent 协作、元认知学习、个性化行为）的基础设施。

参考架构：基于 Logico.md 的五层记忆架构，结合 2024-2025 年研究的工程实践，推荐以下实现方案：

表 4 记忆层参考架构

层级	名称	职责	存储介质	参考实现
L0	原始记录层	保存原始对话、Action 记录、时间戳	SQLite / JSONL	QueryState 的序列化快照
L1	命题抽取层	从文本中抽取结构化命题（事实、偏好、目标）	向量数据库 + 文本	类似 Mem0 的提取流水线
L2	关系组合层	建立命题间的逻辑关系（支持、蕴含、矛盾）	图数据库 / 内存图	类似 A-MEM 的 Zettelkasten
L3	用户画像层	管理实体、概念、用户偏好和长期目标	结构化存储	本体图谱（Entity-Relation）
L4	反思更新层	根据新对话更新对用户和任务的理解	派生自 L0-L3	自我修正的逻辑图像

与现有架构的集成点：

1. **ContextProvider 协议扩展**：新增 `memory_provider` 属性，QueryContext 可选择性地注入 MemoryProvider。MemoryProvider 在 query loop 启动时读取相关记忆，在 loop 结束时将新记忆持久化。
2. **ActionRecordManager 对接 L0**：Action 执行记录天然构成 L0 的原始素材。可在 loop 结束时将 `action_record_list` 批量写入 L0 存储。
3. **新增 memory Action**：支持 LLM 主动调用记忆操作（检索、存储、更新），类似 MemGPT 的 self-directed memory operations。Action profile 定义为：
 - `action_intention`: INTERNAL_REASONING
 - `action_environment_effect`: ADDITIVE（存储）/ READ_ONLY（检索）
 - `llm_dependency`: REQUIRED（命题抽取需要 LLM）
4. **PromptBuilder 自动注入**：在构建 prompt 时，MemoryProvider 自动检索与当前 query 相关的记忆片段，作为 CONTEXT 的一部分注入。支持 `include_context=` `["memory.relevant_facts", "memory.user_preferences"]` 的字段选择语法。

MVP 范围：Phase 1 可先实现 L0 + L1 的简化版——将 `action_record_list` 序列化为 JSONL（L0），通过 LLM 调用抽取关键事实和偏好存储到向量数据库（L1），检索时通过向量相似度 + 时间衰减获取相关记忆。L2-L4 可在后续迭代中逐步引入。

4.2 P0: INTERNAL_CALL 与 Sub-Agent 委托 P0

目标：实现 Sub-Agent 委托机制，使 TinySoul 能够将复杂任务分解为子任务并委托给专门的 Sub-Agent 执行。

为什么优先：当前 TinySoul 的 QueryLoop 是单 Agent 架构——一个 loop 内 LLM 按顺序选择 Action。当任务复杂度上升时，单 Agent 面临两个根本问题：（1）上下文窗口被大量 Action meta/detail 填满；（2）LLM 在过多选项中的选择准确率下降。GRASP 的研究证实，**提供更多技能不会单调提升性能**——聚焦的 2-3 个技能胜过全面的文档。Sub-Agent 委托通过“分而治之”解决这一瓶颈：每个 Sub-Agent 只看到其专业领域的少量 Action，从而保持选择准确率。

设计方案：将 Sub-Agent 实现为一种特殊的 Action（`INTERNAL_CALL`），其核心机制如下：

1. **Sub-Agent 定义**：Sub-Agent 是一个配置对象（非运行时进程），包含：
 - `name + description`（用于父 Agent 的选择决策）
 - `system prompt`（定义 Sub-Agent 的角色和行为边界）
 - `available_actions`（该 Sub-Agent 可使用的 Action 白名单）
 - `max_turns`（Sub-Agent 的最大 turn 数）
 - `output_schema`（Sub-Agent 返回数据的 JSON schema）

2. 执行流程：当父 Agent 选择 `INTERNAL_CALL` 时：

1. 创建新的 QueryLoop 实例，传入子任务描述、Sub-Agent 配置、当前 Context 的只读快照
2. 子 loop 独立运行，使用其专属的 Action 白名单
3. 子 loop 完成后，返回的结果被封装为 action_result 进入父 loop 的 action_record_list
4. 父 loop 在 Step 3 看到 Sub-Agent 的结果后决定下一步

3. 与现有架构的契合：

- 复用现有的 QueryLoop、QueryState、ActionRegistry——无需新增核心组件
- Sub-Agent 的 available_actions 复用 ActionRegistry.with_allowlist() 机制
- Context 的只读快照可通过序列化 + 反序列化 QueryState 实现
- ErrorTrap 的 Disposition 体系自然支持子 loop 的异常传播

设计注意事项

Sub-Agent 的关键设计决策是上下文隔离边界——子 loop 应该看到多少父 loop 的上下文？
推荐方案：子 loop 接收父 loop 的 user_query、workspace 引用和一份当前 state 的静态快照（而非实时同步），但子 loop 的状态变化不反向同步到父 loop。这种"单向数据流"保持了架构的简单性和可预测性，也与 OpenAI Agents SDK 中 handoff 的设计哲学一致。

4.3 P1: Action 技能图谱与结构化编排 P1

目标：将当前扁平的 Action 列表升级为可编译的技能图谱（Skill Graph），引入 precondition-effect 边和 DAG 编译层，实现结构化的 Action 编排。

为什么重要：当前 TinySoul 的 Step 1（choose_action）从可用 Action 中选择一个最优 Action。当可用 Action 数量超过 10 个时，LLM 的选择准确率下降。更重要的是，LLM 难以在单次选择中捕捉 Action 间的因果依赖关系——"必须先 scan_workspace 才能 read_file"这类知识无法通过 Action meta 列表有效传达。GRASP 证明，将技能集编译为带有 precondition-effect 边的 DAG 可以将环境步数减少 41%。

设计方案：

1. **Action Contract 增强：**扩展现有的 action_contract，显式声明 precondition 和 effect：

```
{
  "preconditions": [
    {"type": "state", "field": "workspace.resources", "condition":
"non_empty"},
    {"type": "action_done", "action": "scan_workspace"}
  ],
  "effects": [
    {"type": "state_add", "field": "workspace.resources[].content"},
    {"type": "milestone", "description": "文件已读取"}
  ]
}
```

2. Skill Graph 编译器：在 Step 1 和 Step 2 之间插入编译阶段：

1. LLM 在 Step 1 输出一组候选 Action（而非单一 Action）
2. 编译器根据 precondition-effect 边将候选 Action 编译为 DAG
3. 检查 DAG 的可达性和前置条件满足性
4. 拓扑排序后生成执行计划

3. 与现有架构的契合：

- ActionMeta 中已有 action_contract.preconditions 和 postconditions 字段，只需增强其结构化程度
- 已有的 ParallelDispatcher 可直接用于 DAG 中无依赖节点的并行执行
- ErrorTrap 的双记录机制可用于 DAG 执行中的局部失败处理

渐进路径：不需要一次性实现完整的 DAG 编译。Phase 1 可先引入"Action 依赖提示"——在 Step 1 的 prompt 中，除了 Action meta 列表，还注入一张"Action 依赖关系图"（文本格式），让 LLM 在选择时感知因果关系。Phase 2 再引入正式的编译器。

4.4 P1: MCP 协议适配层 P1

目标：让 TinySoul 可以作为 MCP Host，接入标准化的 MCP Server 生态，动态发现和使用外部工具。

为什么重要：MCP 正在快速成为 Agent 工具连接的事实标准。目前已有数千个社区 MCP Server，覆盖 GitHub、Slack、文件系统、数据库等。为 TinySoul 添加 MCP 支持意味着 Agent 的能力边界可以从"内置 Action"扩展到"整个 MCP 生态"，而无需为每个工具编写适配代码。

设计方案：

1. **MCPAdapter Action**: 实现一组特殊的 Action 作为 MCP 桥接:
 - `mcp_discover`: 发现并列出可用的 MCP Server 和工具
 - `mcp_call`: 调用指定 MCP Server 的工具
2. **ActionRegistry 集成**: MCP Server 提供的工具在运行时被转换为 ActionHandler 实例注册到 ActionRegistry（动态注册机制的复用），使 LLM 可以像调用普通 Action 一样调用 MCP 工具。
3. **安全策略**: MCP 工具的执行需要显式用户授权（至少首次），可通过现有的 `ask_user` Action + 授权缓存机制实现。

与现有架构的契合: TinySoul 已有的动态 Action 注册机制（`register_temporary_script`）和 ActionRegistry 的 `with_allowlist` 过滤，为 MCP 工具的动态注册和权限控制提供了现成基础。MCP 适配层的核心工作量在于 MCP 协议客户端的实现（std-io 或 HTTP 传输层）。

4.5 P1: 元认知反射与经验积累 P1

目标: 建立跨 Query 的元认知反射机制，使 TinySoul 能够从执行经验中提取可复用的策略，并融入未来任务的决策上下文。

为什么重要: 当前 TinySoul 的 reasoning action 是单步的——LLM 在某一 turn 进行反思，但反思结果只影响当前 loop。MetaAgent 的研究表明，将反思经验累积并注入未来任务上下文，可以持续提升 Agent 的表现，且不需要变更模型参数。这与 TinySoul 的"个人 Agent"定位高度契合——个人 Agent 应该在使用中越来越了解用户。

设计方案:

1. **经验存储**: 将 reasoning action 的输出（reflection 内容）作为 L1 记忆的一种特殊类型（"经验记忆"）存储到记忆系统中。每条经验包含:
 - 触发场景（task pattern）
 - 反思内容（lesson learned）
 - 效果验证（success indicator）
 - 使用次数和最后使用时间（用于衰减和淘汰）
2. **经验检索**: 在 query loop 启动时，MemoryProvider 根据当前 query 的语义相似度检索相关经验，作为 system context 的一部分注入。
3. **经验验证**: 建立闭环验证机制——当某条经验被应用后，追踪其效果（任务成功/失败），定期清理被验证为无效的经验。

与现有架构的契合：完全复用 P0 记忆系统的 L0-L1 层。唯一的扩展是在记忆类型中增加"经验"（experience）类别，以及在 PromptBuilder 中增加经验的自动注入逻辑。

4.6 P2：行为草图系统（Behavior Drafts） P2

目标：实现用户可自定义的行为模式——用户可以定义新的 Action 组合模式、决策规则和响应模板，Agent 将其作为"行为草图"加载并在执行中遵循。

为什么放在 P2：行为草图系统是一个高度用户-facing 的功能，其实现依赖于 Sub-Agent（定义子工作流模板）和记忆系统（持久化用户偏好）的先行落地。在底层能力成熟前，行为草图的实现将是脆弱且难以维护的。

初步思路：行为草图可以被视为"用户定义的 Sub-Agent 模板"——用户通过自然语言或结构化格式描述一组行为规则（如"每当我提到写代码时，先检查 workspace 中是否有相关文件"），系统将其编译为 Sub-Agent 配置 + 约束条件，在执行特定类型任务时自动加载。

五、实施路线图

5.1 Phase 1: 记忆层 MVP（4-6 周）

Phase 1 · 优先级 P0 · 记忆系统 + Sub-Agent 基础

表 5 Phase 1 任务分解

周次	任务	产出	依赖
W1	L0 层实现: action_record_list 序列化为 JSONL	tinysoul/memory/l0_store.py	无
W1-2	L1 层实现: 事实抽取 + 向量存储	tinysoul/memory/l1_extractor.py, l1_store.py	L0
W2-3	MemoryProvider 协议 + QueryContext 集成	ContextProvider 新增 memory 属性	L1
W3-4	memory Action 实现 (检索/存储)	新增内置 Action 2 个	MemoryProvider
W4-5	INTERNAL_CALL Action 实现 (Sub-Agent 委托)	tinysoul/action/handlers/internal_call.py	无
W5-6	Sub-Agent 配置系统 + 上下文隔离	SubAgentConfig + 序列化快照	INTERNAL_CALL
W6	集成测试 + 文档	测试覆盖 >80%, 用户文档	全部

Phase 1 完成标准: Agent 能够在两次对话间记住用户的偏好和项目信息; 能够将复杂任务委托给配置好的 Sub-Agent; 记忆系统对 LLM 透明 (通过 Action 调用), 对上层代码通过 Protocol 访问。

5.2 Phase 2: Sub-Agent 框架与技能图谱（4-6 周）

Phase 2 · 优先级 P1 · Sub-Agent 增强 + Action 编排

表 6 Phase 2 任务分解

周次	任务	产出	依赖
W1-2	Action Contract 增强: precondition-effect 结构化	更新的 ACTION_JSON schema	无
W2-3	Action 依赖图 (文本格式) 注入 Step 1 prompt	PromptBuilder 增强	Contract 增强
W3-4	Sub-Agent 模板库 (预定义的专业 Agent 配置)	tinysoul/subagents/templates/	Phase 1 Sub-Agent
W4-5	MCP 适配层 MVP (stdio 传输)	tinysoul/mcp/ 模块	无
W5-6	元认知反射: 经验存储与检索	reasoning action 增强 + 经验类型	Phase 1 记忆系统

5.3 Phase 3: 编排与生态 (6-8 周)

Phase 3 · 优先级 P2 · 行为草图 + DAG 编译 + MCP 完善

表 7 Phase 3 任务分解

周次	任务	产出	依赖
W1-3	DAG 编译器: 从候选 Action 到类型化执行图	tinysoul/planning/dag_compiler.py	Phase 2 Contract
W3-5	局部修复: DAG 节点失败时的 bounded replan	修复算子实现	DAG 编译器
W5-6	MCP 远程传输 + 安全策略完善	HTTP transport + 授权 UI	Phase 2 MCP
W6-8	行为草图系统 MVP	自然语言到 Sub-Agent 模板	Phase 2 模板库

六、技术选型建议

表 8 关键技术选型建议

组件	推荐方案	备选方案	决策理由
L0 存储	JSONL 文件（本地优先）	SQLite	TinySoul 是个人 Agent，本地文件优先，避免外部依赖
L1 向量存储	嵌入式向量库（如 <code>sqlite-vec</code> ）	Chroma, Qdrant	零外部依赖，单文件存储
L2 图存储	内存图 + 序列化（NetworkX）	Kuzu, Neo4j	Phase 1-2 不需要持久化图，内存足够
MCP 传输	stdio（本地）+ Streamable HTTP（远程）	SSE（旧版）	遵循 MCP 2025-11 规范最新标准
经验存储格式	结构化 JSON + 文本	纯文本	支持按字段过滤（如按 <code>task_type</code> ）

核心原则：TinySoul 作为个人 Agent，技术选型应遵循"本地优先、最小依赖、单文件存储"的原则。向量存储选 `sqlite-vec`（单文件 SQLite 扩展），图存储选内存 NetworkX（序列化为 JSON），避免引入需要独立进程的数据库服务。这与当前 TinySoul 仅依赖 `openai` 包和 Python 标准库的设计哲学一致。

七、风险与挑战

表 9 主要风险与缓解策略

风险	影响	缓解策略
记忆系统引入过多 LLM 调用，增加延迟和成本	高	L0 不经过 LLM（直接序列化）；L1 提取可在后台异步进行；引入缓存层避免重复提取
Sub-Agent 的递归调用导致无限循环	高	限制委托深度（max_depth=2）；总 turn 数配额机制；父 loop 可中断子 loop
MCP 工具的安全风险（恶意 Server）	高	首次使用需用户授权；沙箱执行 MCP 工具调用；只读操作默认允许，写操作需二次确认
DAG 编译增加系统复杂度	中	渐进引入——先依赖提示，再轻量编译，最后完整 DAG；提供降级路径（fallback 到现有扁平选择）
记忆质量低（错误事实、过时偏好）	中	每条记忆附带置信度和时间戳；定期清理低置信度记忆；用户可通过对话纠正记忆
经验积累过慢（个人 Agent 使用频率有限）	低	经验来源不限于成功/失败，还包括"有效的工作流程模式"；支持手动注入经验

八、结论

TinySoul 已经构建了一个工程成熟度极高的 Agent 核心——清晰的模块边界、精心设计的运行时状态、OS-Style 异常路由、安全的脚本沙箱、多模型故障转移。这些基础能力为下一步演进提供了坚实的平台。

本报告基于对七份设计文档的深度阅读和对 2024-2025 年 Agent 领域前沿研究的系统调研，提出 TinySoul 的下一步演进应围绕三个核心主题展开：

1. 记忆——从无状态到有状态。通过跨 Query 记忆系统的引入，TinySoul 将从"每次对话从零开始"进化为"积累对用户和任务的理解"。这不仅是功能增强，更是 Agent 从"工具"到"伙伴"的关键跃迁。Logico.md 中提出的五层记忆架构和命题化记忆模型，与 A-MEM、Mem0、GRASP 等前沿研究高度一致，具备坚实的理论基础和实践可行性。

2. 委托——从单 Agent 到分层协作。通过 Sub-Agent 委托机制，TinySoul 将能够处理更复杂的任务，同时保持每个决策节点的选择准确率。INTERNAL_CALL 的占位符设计表明这一方向早在架构初期就被预见，现在条件已经成熟。

3. 演进——从静态配置到动态学习。通过元认知反射和经验积累，TinySoul 将在不变更模型参数的情况下持续改进其行为策略。MetaAgent 的研究表明这种"learning-by-doing"的方法是可行的，且效果显著。

这三个主题之间存在深层依赖关系：记忆为委托提供上下文连续性，委托为元认知提供多样化的经验来源，元认知的产出又丰富记忆系统的内容。它们共同构成一个正反馈循环，驱动 TinySoul 从一个"执行指令的 Agent"演进为一个"理解用户、记住偏好、持续改进的个人 Agent"。

实施上，建议按 Phase 1（记忆层 MVP + Sub-Agent 基础，4-6 周）→ Phase 2（技能图谱 + MCP + 元认知，4-6 周）→ Phase 3（DAG 编译 + 行为草图，6-8 周）的路线图分阶段推进，每阶段都有明确的完成标准和可交付成果。

参考文献

1. Packer, C., Fang, V., Patil, S. G., Lin, K., Wooders, S., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*.
2. Packer, C., et al. (2024). Letta: Stateful agents with persistent memory and tool use. *Letta Technical Report*.
3. Xu, Y., Liang, J., Mei, J., Gao, Y., Tan, M., & Zhang, Y. (2025). A-MEM: Agentic memory with self-organizing Zettelkasten structure. *arXiv preprint*.
4. Chhikara, P., Khant, D., Aryan, S., Singh, T., & Yadav, D. (2025). Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*.
5. Xia, T., Hu, L., Sun, Y., Xu, M., Xu, L., Wang, S., Xu, W., & Jiang, J. (2026). GraSP: Graph-structured skill compositions for LLM agents. *arXiv preprint arXiv:2604.17870*.
6. Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., & Wang, Y.-X. (2024). Language agent tree search unifies reasoning, acting, and planning in language models. *ICML*.
7. Qian, H., et al. (2025). MetaAgent: Toward self-evolving agent via tool meta-learning. *arXiv preprint arXiv:2508.00271*.
8. Anthropic. (2024). Model Context Protocol (MCP) specification. <https://modelcontextprotocol.io>.
9. Zhong, W., Guo, L., Gao, Q., Ye, H., & Wang, S. (2024). MemoryBank: Enhancing LLMs with long-term memory. *arXiv preprint*.
10. Shi, Y., et al. (2025). OxyGent: Making multi-agent systems modular, observable, and evolvable via Oxy abstraction. *arXiv preprint arXiv:2604.25602*.
11. OpenAI. (2025). OpenAI Agents SDK. <https://github.com/openai/openai-agents-python>.
12. CrewAI. (2025). CrewAI: Framework for orchestrating autonomous AI agents. <https://www.crewai.com>.
13. Zhang, C., et al. (2024). A survey on memory mechanisms for LLM agents. *arXiv preprint*.
14. Wittgenstein, L. (1921). *Tractatus Logico-Philosophicus*. (Referenced in Logico.md as theoretical foundation).